

EMBEDDED SYSTEMS PROJECT REPORT

Temperature-Controlled Fan Speed Regulation

Using STM32F407VGT6 Microcontroller with PID Control

Rushikesh Lakadkar

M.Sc. Embedded Systems
Technische Universität Chemnitz
Chemnitz, Germany — May 2026

Hardware Platform

STM32F407G-DISC1 Discovery Board | STM32CubeIDE | HAL Drivers
LM35 Temperature Sensor | MOSFET Fan Driver | RGB LED Indicators

Abstract

This report presents the design, implementation, and validation of an embedded temperature-controlled fan speed regulation system built on the STM32F407VGT6 microcontroller. The system integrates multiple hardware peripherals including ADC, PWM, UART, GPIO, and Timer subsystems to achieve closed-loop thermal management using a Proportional-Integral-Derivative (PID) control algorithm.

The LM35 precision temperature sensor provides continuous ambient temperature feedback through a 12-bit ADC conversion. Based on the measured temperature, the system dynamically adjusts a DC fan motor speed via PWM-controlled MOSFET switching, while simultaneously driving three color-coded LEDs (green, yellow, red) to provide a visual indication of the current thermal zone. Serial output over UART enables real-time monitoring and debugging via a host terminal.

The project demonstrates proficiency across the full embedded development stack: peripheral configuration using STM32 HAL libraries, real-time control algorithm implementation in C, hardware-software integration on a breadboard prototype, and iterative debugging using breakpoints, live register and variable inspection, and SWV ITM trace. Results confirm stable fan speed regulation with smooth thermal zone transitions and reliable LED feedback.

Table of Contents

1. Introduction	4
2. System Overview.....	5
3. Hardware Components & Specifications	6
4. Circuit Design & Wiring	11
5. STM32CubeIDE Project Configuration	12
6. Peripheral Driver Implementation	16
7. PID Control Algorithm	18
8. Temperature Zone Logic & LED Control	20
9. Debugging Methodology & Verification.....	21
10. UART Serial Monitoring	25
11. Software Architecture & Code Walkthrough	26
12. Testing & Results.....	28
13. Challenges & Solutions	29
14. Conclusion & Future Work	30
References	31

1. Introduction

Thermal management is a critical aspect of embedded and industrial electronic system design. Uncontrolled heat accumulation in electronic assemblies can significantly reduce component lifespan, degrade performance, and in extreme cases lead to catastrophic failure. Automated fan speed control systems represent an elegant and energy-efficient solution to this challenge, adjusting cooling intensity in real time based on sensed temperature rather than operating at maximum speed continuously.

This project implements a complete embedded thermal regulation system targeting the STM32F407G-DISC1 evaluation board, which is built around the STM32F407VGT6 ARM Cortex-M4 microcontroller running at 84 MHz. The system reads ambient temperature via an LM35 analog sensor, processes the reading through a PID control loop, and drives a DC fan motor at a proportional speed using PWM-modulated MOSFET switching. Three LEDs provide a discrete visual temperature zone display.

1.1 Objectives

- Read ambient temperature continuously using the LM35 analog temperature sensor via ADC Channel 0.
- Convert the ADC digital value to a calibrated temperature in degrees Celsius.
- Implement a PID control algorithm that computes an appropriate fan speed output.
- Drive a DC brushless fan via MOSFET using PWM output from TIM2 Channel 1.
- Indicate thermal zones (low, medium, high) using green, yellow, and red LEDs on GPIO Port D.
- Transmit real-time telemetry (temperature, ADC value, voltage, error, PWM) over USART2 at 115200 baud.
- Demonstrate software tracing using SWV ITM Console in STM32CubeIDE, supported by breakpoint and register-level debugging during development.

1.2 Scope

The scope of this project is limited to ambient air temperature sensing and fan speed regulation. The system is not designed for liquid cooling or high-power industrial applications, but the control architecture is directly transferable to such scenarios. The hardware prototype operates on a breadboard with discrete components to facilitate inspection and iterative debugging.

2. System Overview

The temperature-controlled fan system consists of five functional blocks that interact in a closed-loop feedback architecture. The block diagram below describes the overall signal flow.

Block	Component	Interface	Function
Sensing	LM35DZ Sensor	ADC CH0 (PA0)	Convert ambient temperature to 0–3.3V analog voltage
Signal Conditioning	STM32 ADC1	12-bit SAR, 3 cycles	Digitize analog voltage with 0.8mV resolution
Control	PID Algorithm	Software (C)	Compute error-corrective PWM duty cycle
Actuation	MOSFET + Fan	TIM2 CH1 PWM (PA5)	Drive DC fan at variable speed (30%–90%)
Indication	3x LEDs	GPIO PD12/13/14	Display thermal zone: Green/Yellow/Red
Monitoring	USART2	UART TX (PA2)	Stream telemetry to host terminal at 115200 baud

The system operates in a polling loop with a 1-second period. Each iteration performs an ADC conversion, evaluates the PID controller, applies the PWM output, updates LED indicators, and conditionally transmits a UART log message if the temperature has changed by more than 0.1 °C since the last report.

2.1 Control Loop Summary

The core control loop follows this sequence in each 1-second iteration:

- Trigger ADC conversion on Channel 0 (PA0).
- Wait for conversion completion via HAL polling.
- Read 12-bit ADC value and convert to voltage: $V = (ADC_val \times 3.3) / 4095$.
- Convert voltage to temperature: $T = V \times 100$ (LM35 produces 10 mV/°C).
- Compute PID error terms: proportional, integral, derivative.
- Clamp PID output to valid PWM range [0, 1000].
- Override PID with discrete zone PWM values based on thermal zone boundaries.
- Update TIM2 compare register to set PWM duty cycle.
- Toggle corresponding LED and suppress others.
- Conditionally print UART telemetry if temperature changed.

3. Hardware Components & Specifications

3.1 Microcontroller: STM32F407VGT6

The STM32F407VGT6 is a high-performance ARM Cortex-M4 32-bit RISC microcontroller manufactured by STMicroelectronics. It features a floating-point unit (FPU), DSP instructions, and a rich set of on-chip peripherals, making it well-suited for real-time control and signal processing applications.

Parameter	Specification
Core	ARM Cortex-M4 with FPU
Clock Speed	Up to 168 MHz (configured at 84 MHz in this project)
Flash Memory	1 MB
SRAM	192 KB
ADC	3x 12-bit SAR ADC, up to 2.4 MSPS
Timers	2x Advanced, 4x General Purpose, 2x Basic
UART/USART	6x USART
SPI	3x SPI
I2C	3x I2C
CAN	2x CAN 2.0B
GPIO	Up to 140 I/O pins
Operating Voltage	1.8V to 3.6V
Package	LQFP-100

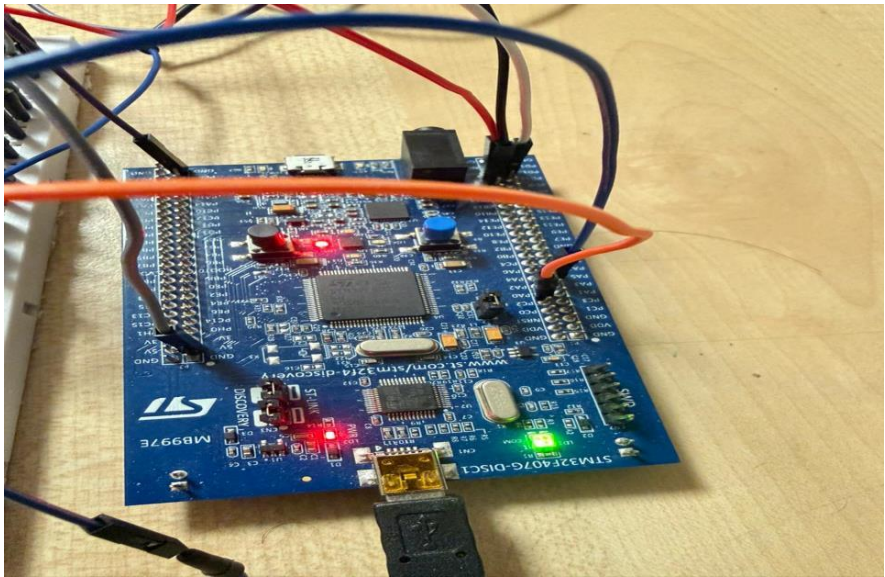


Figure 3.1 – STM32F407G-DISC1 Discovery Board with active connections

3.2 Temperature Sensor: LM35DZ

The LM35 is a precision integrated-circuit temperature sensor from Texas Instruments. Its output voltage is linearly proportional to the Celsius temperature at a sensitivity of exactly 10 mV per degree Celsius. No external calibration is required.

Parameter	Value
Supply Voltage	4V to 30V (operated at 3.3V in this design)
Output Sensitivity	10 mV / °C
Temperature Range	0 °C to +100 °C (LM35DZ variant)
Accuracy	±0.5 °C (typical at 25 °C)
Quiescent Current	60 µA
Output Impedance	0.1 Ω at 1 mA load
Package	TO-92

At room temperature (25 °C), the sensor outputs 250 mV. This scales linearly: 0 °C = 0 V, 50 °C = 500 mV, 100 °C = 1.0 V. The STM32 ADC with 3.3V reference digitizes this with a resolution of approximately 3.3V / 4096 steps = 0.806 mV/step, corresponding to a temperature resolution of ~0.08 °C.

3.3 DC Fan & MOSFET Driver

A standard 5V DC brushless cooling fan (commonly used in PC chassis) is driven via an N-channel MOSFET transistor. The MOSFET gate is connected to the PWM output (TIM2 CH1 on PA5) through a current-limiting gate resistor (typically 100 Ω). A flyback diode (1N4007 or equivalent) is placed in parallel with the fan motor to suppress inductive voltage spikes that could damage the MOSFET during switching.

Component	Role	Key Parameter
DC Fan (5V)	Cooling actuator	Operates at 5V, PWM-controllable via MOSFET
N-MOSFET	High-side switch	Logic-level gate drive from 3.3V STM32 GPIO
Gate Resistor (100 Ω)	Limit gate current	Damps oscillation on gate trace
Flyback Diode	Transient suppression	Blocks inductive kickback from motor winding

3.4 LED Indicators

Three discrete through-hole LEDs are used as thermal zone indicators. Each LED is connected in series with a current-limiting resistor (~220 Ω) between the STM32 GPIO output and GND. The on-board LEDs at PD12, PD13, and PD14 are used directly.

GPIO Pin	LED Color	Thermal Zone	PWM Duty Cycle
PD12	Green	Low (< 26 °C)	30% (PWM = 300)

GPIO Pin	LED Color	Thermal Zone	PWM Duty Cycle
PD13	Yellow (Orange)	Medium (27–28 °C)	60% (PWM = 600)
PD14	Red	High (> 28 °C)	90% (PWM = 900)

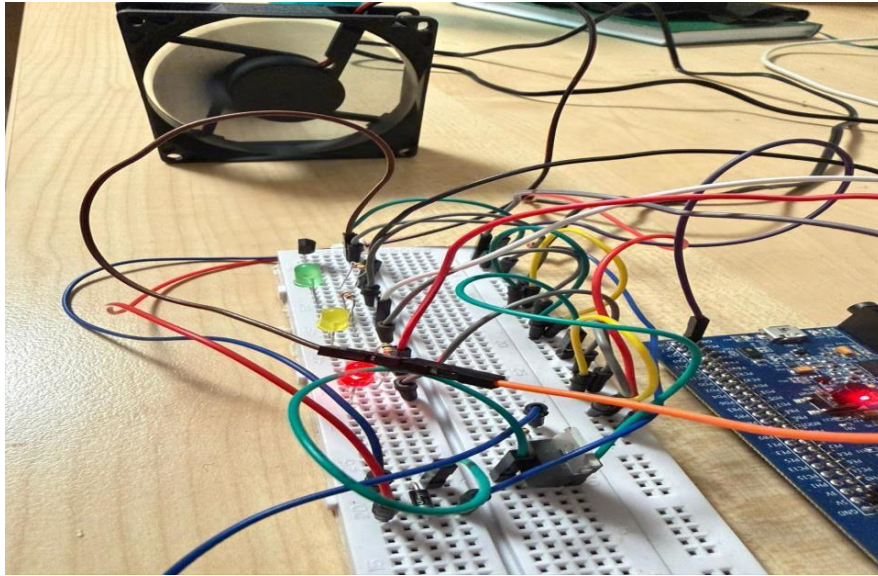


Figure 3.2 – Breadboard prototype showing LED indicators, MOSFET driver circuit, and fan

3.5 Complete System Schematic

Figure 3.3 below consolidates every subsystem described in Sections 3.1–3.4 into a single schematic, showing the STM32F407G-DISC1 (U1), the LM35DZ temperature sensor (U2), and the MOSFET fan driver stage (U3), together with the LED indicator bank, the J1 Micro-USB power/data connector, and the ST-LINK/UART breakout used for host-PC monitoring.

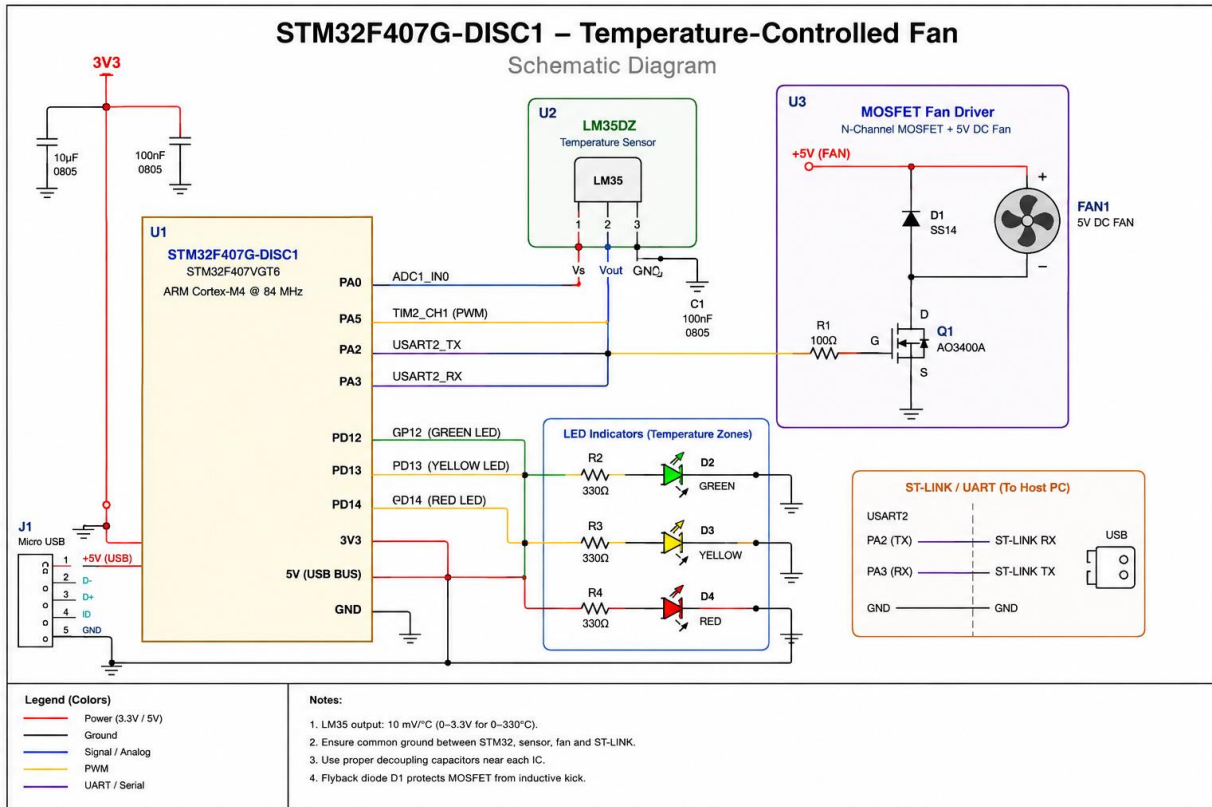


Figure 3.3 – Complete schematic of the STM32F407G-DISC1 temperature-controlled fan system

3.5.1 Power Distribution

The 3.3V rail from the Discovery board's on-board regulator supplies U1's logic, the LM35 sensor (U2 pin 1, Vs), and is decoupled locally with a 10 μ F bulk capacitor and a 100 nF ceramic capacitor at the supply entry point, as shown at the top-left of Figure 3.3. The fan circuit (U3) instead runs from the separate +5V (FAN) rail sourced from the USB bus via J1 pin 1, keeping the fan's switching current off the 3.3V analog rail that feeds the LM35 — this separation is what keeps the ADC reading clean while the MOSFET is switching at 1 kHz.

3.5.2 Signal Routing

PA0 (blue, analog) connects directly to the LM35's Vout pin (U2 pin 2), with a 100 nF capacitor (C1) at the sensor output node to filter high-frequency noise before the signal reaches the ADC. PA5 (yellow, PWM) runs from TIM2_CH1 to the gate of Q1 through the 100 Ω gate resistor (R1), exactly as configured in Section 5.3. PA2/PA3 (purple, UART) branch in two directions: down to the ST-LINK/UART breakout for host-PC monitoring, and across to the LED indicator bank's shared signal node, consistent with the USART2 wiring in Section 4.1.

3.5.3 LED and Fan Stages

PD12, PD13, and PD14 (green) each drive one LED through its own 330 Ω series resistor (R2, R3, R4) directly to ground, matching the zone assignments in Section 3.4. On the fan side, the flyback diode D1 (SS14) sits across FAN1, clamping the inductive spike generated when Q1 (an AO3400A N-channel MOSFET) switches off — this is the same protection role described for the 1N4007 diode in Section 4.3, with D1 here being a faster Schottky alternative suited to the 1 kHz switching frequency.

3.5.4 System in Operation

Figure 3.4 shows the assembled breadboard prototype running, with the green and red zone LEDs both visible and the cooling fan connected and energised. This corresponds to the physical realisation of the schematic in Figure 3.3 and the wiring summary in Section 4.1.

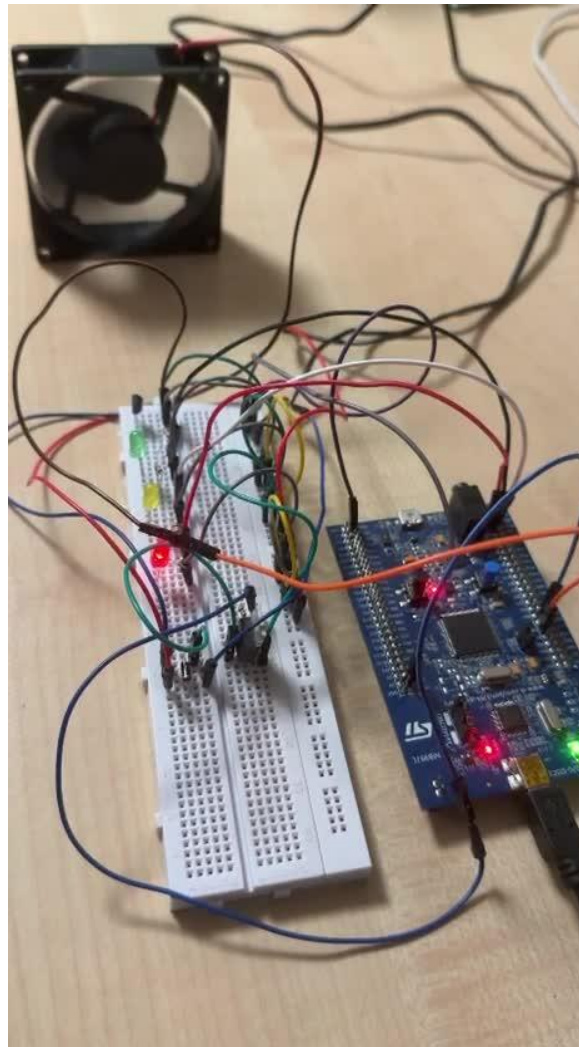


Figure 3.4 – Assembled prototype during operation: breadboard wiring, LED indicators, and cooling fan

4. Circuit Design & Wiring

The full schematic for this wiring is presented as Figure 3.3 in Section 3.5; the tables below summarise the same connections in tabular form for quick reference during assembly and debugging.

4.1 Pin Assignment Summary

STM32 Pin	Function	Peripheral	Connected To
PA0	ADC1_IN0	ADC Channel 0	LM35 Vout pin
PA5	TIM2_CH1	PWM Output	MOSFET Gate (via 100 Ω)
PA2	USART2_TX	UART Transmit	USB–UART bridge (ST-LINK)
PA3	USART2_RX	UART Receive	USB–UART bridge (ST-LINK)
PD12	GPIO_OUTPUT	LED Green	Green LED anode (via 220 Ω)
PD13	GPIO_OUTPUT	LED Yellow	Yellow LED anode (via 220 Ω)
PD14	GPIO_OUTPUT	LED Red	Red LED anode (via 220 Ω)
GND	Power	Common Ground	LM35 GND, MOSFET Source, LED cathodes
3.3V	Power	Supply	LM35 Vs pin
5V	Power	Fan Supply	Fan positive terminal

4.2 LM35 Sensor Wiring

The LM35 TO-92 package has three pins: Vs (power), Vout (analog output), and GND. In this project, Vs is connected to the STM32 3.3V supply rail, GND to common ground, and Vout to PA0. A 100 nF ceramic decoupling capacitor is recommended between Vs and GND close to the sensor to filter supply noise from affecting the output voltage.

4.3 MOSFET Fan Driver Circuit

The N-channel MOSFET source is tied to GND. The drain connects to the fan negative terminal. The fan positive terminal connects to the 5V rail. A 1N4007 freewheeling diode is placed cathode-to-5V and anode-to-drain (across the fan). The gate is driven from PA5 (PWM output) through a 100 Ω series gate resistor. This limits the gate charging current and prevents ringing on the gate signal trace.

4.4 Power Considerations

The STM32F407G-DISC1 board is powered via its USB connector (5V bus). The on-board 3.3V LDO regulator supplies the MCU and is used here to power the LM35. The fan motor draws current from the USB 5V bus through the MOSFET. Care must be taken that total current draw (fan + MCU) does not exceed the host USB port limit (500 mA for USB 2.0). A separate 5V supply is recommended for fan loads above 400 mA.

5. STM32CubeIDE Project Configuration

The project was created and configured using STM32CubeIDE version 1.x with integrated CubeMX for peripheral initialization code generation. The following subsections detail each peripheral configuration.

5.1 System Clock Configuration

The system clock is derived from the internal HSI oscillator (16 MHz) via the PLL. The PLL is configured to produce an 84 MHz SYSCLK, which drives the AHB bus at 84 MHz. APB1 is divided by 2 (42 MHz), and APB2 runs at 84 MHz.

Clock Parameter	Setting	Resulting Frequency
Oscillator	HSI (internal RC)	16 MHz
PLL Source	HSI	16 MHz input
PLLM	16	1 MHz VCO input
PLLN	336	336 MHz VCO output
PLL P	DIV4	84 MHz SYSCLK
AHB Prescaler	DIV1	84 MHz HCLK
APB1 Prescaler	DIV2	42 MHz (TIM2 clock = 84 MHz)
APB2 Prescaler	DIV1	84 MHz
Flash Latency	2 Wait States	Required for 84 MHz operation

Since APB1 prescaler is 2, the timer clock multiplier doubles the APB1 frequency, resulting in TIM2 running at 84 MHz. This is important for the PWM frequency calculation.

Figure 5.1 shows the resulting clock tree as rendered by the STM32CubeIDE Clock Configuration view. The HSI RC oscillator (16 MHz, highlighted blue) feeds the Main PLL through the /16 M-divider and the $\times N$, /P stages, producing the 84 MHz SYSCLK seen at the System Clock Mux output. From there, the AHB Prescaler (/1) yields 84 MHz HCLK, which fans out to the Cortex System timer, the FCLK, and both APB prescaler stages. The APB1 Prescaler (/2) produces a 42 MHz PCLK1, while the adjacent $\times 2$ timer-clock multiplier restores the APB1 timer clock to 84 MHz — exactly the TIM2 input clock used in the Section 5.3 calculation. The APB2 Prescaler (/1) leaves PCLK2 and its timer clock at 84 MHz.

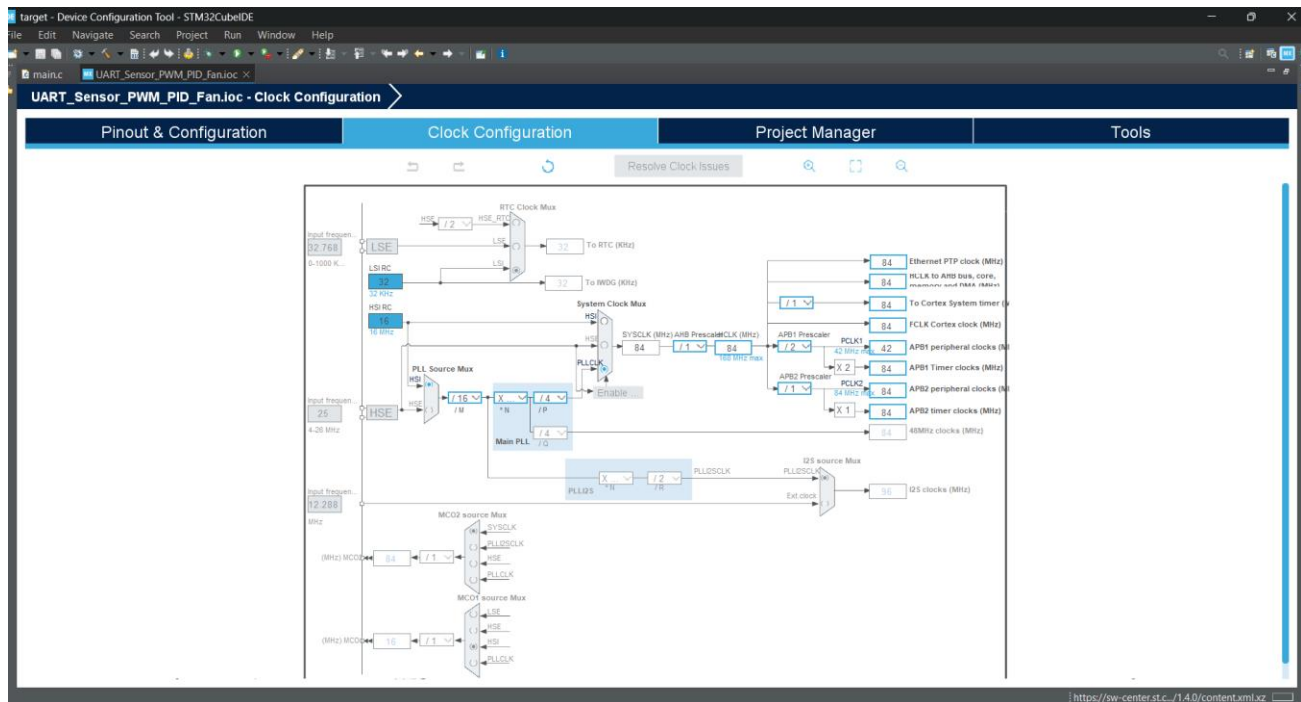


Figure 5.1 – STM32CubeIDE Clock Configuration view showing the HSI → PLL → 84 MHz SYSCLK derivation and APB1/APB2 prescaler outputs

5.2 ADC1 Configuration

ADC Parameter	Setting
Instance	ADC1
Clock Prescaler	PCLK2 divided by 4 (21 MHz ADC clock)
Resolution	12-bit
Scan Mode	Disabled (single channel)
Continuous Mode	Disabled
Trigger	Software start
Data Alignment	Right-aligned
Sampling Time	3 cycles
Channel	ADC_CHANNEL_0 (PA0)
EOC Selection	Single conversion

5.3 TIM2 PWM Configuration

TIM2 is configured as a PWM generator with a 1 kHz PWM frequency and 1000-step resolution (period = 1000, allowing 0–1000 PWM compare values corresponding to 0–100% duty cycle).

TIM2 Parameter	Value	Calculation
Timer Clock	84 MHz	APB1 x2 due to APB1 prescaler != 1
Prescaler	$84 - 1 = 83$	Timer input = $84 \text{ MHz} / 84 = 1 \text{ MHz}$
Period (ARR)	$1000 - 1 = 999$	PWM frequency = $1 \text{ MHz} / 1000 = 1 \text{ kHz}$
PWM Mode	PWM Mode 1	Output high when CNT < CCR
Polarity	Active High	High PWM = higher duty cycle
Initial Pulse	0	Fan starts at 0% duty cycle
Channel	TIM_CHANNEL_1 (PA5)	Alternate function AF01

5.4 USART2 Configuration

USART2 Parameter	Value
Baud Rate	115200 bps
Word Length	8 bits
Stop Bits	1
Parity	None
Mode	TX and RX
Hardware Flow Control	None
Oversampling	16x

5.5 GPIO Configuration

GPIO Port D pins 12, 13, and 14 are configured as push-pull digital outputs with no pull-up/pull-down resistors and low-speed I/O frequency setting. Port A must have the RCC clock enabled for PA0 (ADC), PA2 (UART TX), and PA5 (TIM2 CH1 AF01).

Figure 5.2 shows the resulting pin assignment as rendered in the STM32CubeIDE Pinout view for the STM32F407VGTx LQFP100 package. PD12, PD13, and PD14 are labelled GPIO_Output (green) on the right edge of the package, confirming the LED driver configuration in Section 3.4. PA0 is shown in analog yellow and annotated ADC1_IN0 at the lower-left, matching the LM35 sensor input from Section 4.1. PA2 and PA3, at the bottom edge, are labelled USART2_TX and USART2_RX respectively, and PA5 alongside them is labelled TIM2_CH1, confirming the PWM output pin used for the MOSFET gate drive in Section 5.3.

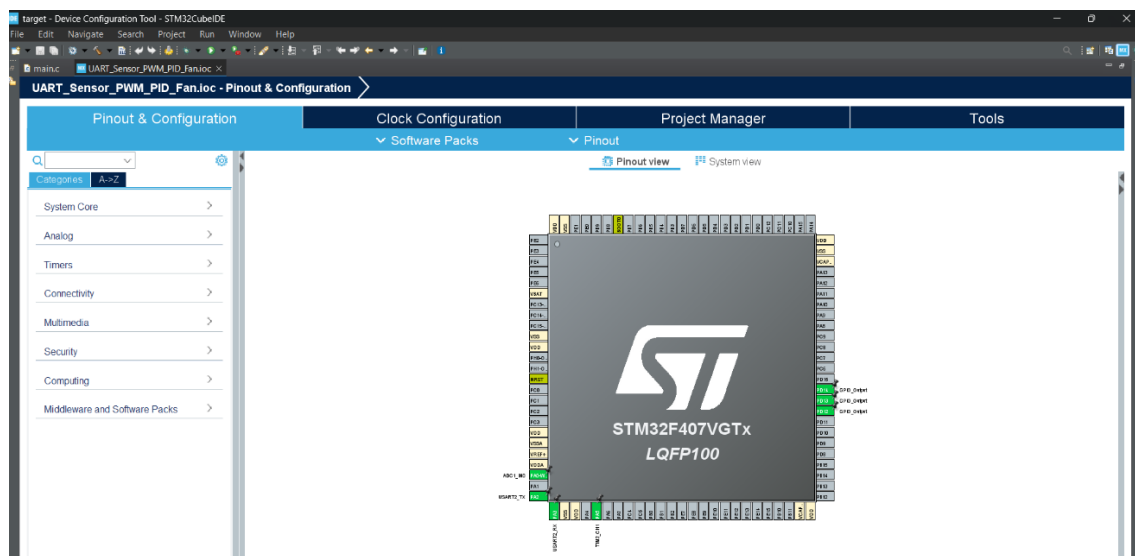


Figure 5.2 – STM32CubeIDE Pinout view of the STM32F407VGTx (LQFP100), showing ADC1_IN0 (PA0), USART2_TX/RX (PA2/PA3), TIM2_CH1 (PA5), and the three GPIO_Output LED pins (PD12–PD14)

6. Peripheral Driver Implementation

6.1 ADC Polling Driver

The ADC is operated in single-conversion polling mode. The `HAL_ADC_Start()` function initiates a conversion, and `HAL_ADC_PollForConversion()` blocks until the conversion is complete. A 5 ms delay is inserted before polling to allow the ADC to settle. This approach is straightforward and appropriate for a 1-second sampling interval.

```
HAL_ADC_Start(&hadc1);
HAL_Delay(5);
HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
adc_val = HAL_ADC_GetValue(&hadc1);
voltage = (adc_val * 3.3f) / 4095.0f;
temperature = voltage * 100.0f; // LM35: 10mV/degC
```

The voltage conversion uses the 3.3V reference voltage and the 12-bit ADC range (0–4095). The temperature conversion exploits the LM35 transfer function directly: since $1\text{V} = 100\text{ }^{\circ}\text{C}$ (at $10\text{ mV}/^{\circ}\text{C}$), multiplying voltage by 100 yields temperature in Celsius.

6.2 TIM2 PWM Output Driver

Timer 2 is initialized and the PWM channel is started using `HAL_TIM_PWM_Start()`. The duty cycle is updated at runtime by writing to the capture-compare register using the `__HAL_TIM_SET_COMPARE` macro. This provides efficient single-register update without re-initializing the timer.

```
// Initialization (called once in USER CODE BEGIN 2):
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);

// Runtime PWM update (called each loop iteration):
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, (uint32_t)pwm_output);
```

6.3 GPIO LED Driver

LED control uses `HAL_GPIO_WritePin()` to reset all three LED pins simultaneously, followed by `HAL_GPIO_TogglePin()` on the active zone LED. Toggling rather than writing produces a visual blinking effect at 1 Hz (matching the loop period), providing a livelier indication than a static LED. The other two LEDs are explicitly held low.

```
// Turn OFF all LEDs at start of each iteration:
HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12|GPIO_PIN_13|GPIO_PIN_14, GPIO_PIN_RESET);
// Toggle the active zone LED:
HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_12); // Green: low zone
```

6.4 USART2 printf Retargeting

The standard C library `printf()` function is retargeted to the SWV ITM trace output by overriding the `_write()` system call. Each character is sent via the `ITM_SendChar()` function, which writes to the Instrumentation Trace Macrocell (ITM) stimulus port. This allows `printf` output to appear in the SWV ITM Data Console in STM32CubeIDE without occupying a UART peripheral.

```
int _write(int file, char *ptr, int len) {
    for (int i = 0; i < len; i++) {
        ITM_SendChar(*ptr++);
    }
    return len;
}
```

Note: To enable SWV output, the Debug configuration must have Serial Wire Viewer enabled with the correct core clock frequency (84 MHz) and ITM stimulus port 0 enabled.

6.5 Additional Peripheral Capabilities (UART, SPI, I2C, CAN)

While this project's primary implementation uses ADC, TIM/PWM, GPIO, and SWV ITM, the STM32F407VGT6 platform and HAL library fully support UART, SPI, I2C, and CAN peripherals. The following provides an overview of each:

UART (USART2)

USART2 is initialized at 115200 baud and is physically routed through the ST-LINK USB interface on the DISC1 board. It can be used for bidirectional communication with a host PC using `HAL_UART_Transmit()` and `HAL_UART_Receive()`. In this project it is available but `printf` is routed through ITM. An alternative `_write()` implementation using `HAL_UART_Transmit()` was considered and can be enabled trivially.

SPI

The STM32F407 includes three SPI peripherals. SPI is used for high-speed synchronous communication with external devices such as DAC chips, display drivers, or external EEPROM. `HAL_SPI_Transmit()`, `HAL_SPI_Receive()`, and `HAL_SPI_TransmitReceive()` provide polling, interrupt, and DMA transfer modes.

I2C

Three I2C peripherals are available for interfacing with sensors and actuators that use the two-wire protocol. Common use cases include reading from digital temperature sensors (e.g., MCP9808), writing to OLED displays, or communicating with IMU modules (e.g., MPU-6050). `HAL_I2C_Master_Transmit()` and `HAL_I2C_Mem_Read()` are the primary HAL functions.

CAN Bus

The STM32F407 contains two `bxCAN` controllers compliant with CAN 2.0B. CAN bus is heavily used in automotive and industrial embedded systems for multi-node communication over a differential bus. `HAL_CAN_Start()`, `HAL_CAN_AddTxMessage()`, and `HAL_CAN_GetRxMessage()` enable reliable frame exchange at bit rates up to 1 Mbit/s.

7. PID Control Algorithm

7.1 PID Theory

A Proportional-Integral-Derivative (PID) controller is a feedback control mechanism that continuously calculates an error value as the difference between a desired setpoint and a measured process variable, then applies a correction based on three terms:

- Proportional (P): Correction proportional to the current error. Large K_p provides fast response but may cause overshoot.
- Integral (I): Correction based on the accumulated error over time. Eliminates steady-state offset but can cause windup if not managed.
- Derivative (D): Correction based on the rate of change of the error. Damps overshoot and improves settling time.

The discrete-time PID equation implemented in this project is:

$$\text{output}(n) = K_p \times e(n) + K_i \times \text{SUM}[e(n)] + K_d \times [e(n) - e(n-1)]$$

7.2 PID Parameter Selection

Parameter	Symbol	Value	Rationale
Setpoint	SP	25.0 °C	Target ambient temperature for fan activation
Proportional Gain	K_p	5.0	Moderate gain; 1 °C error produces 5 PWM units correction
Integral Gain	K_i	0.1	Slow integral wind-up to handle steady offset without oscillation
Derivative Gain	K_d	1.0	Moderate damping to reduce overshoot
Max PWM	pwm_max	1000	Maps to 100% duty cycle (TIM2 Period = 1000)

7.3 Code Implementation

```
float error = setpoint - temperature;
integral += error;
float derivative = error - prev_error;
pwm_output = Kp * error + Ki * integral + Kd * derivative;

// Anti-windup clamp:
if (pwm_output > pwm_max) pwm_output = pwm_max;
if (pwm_output < 0) pwm_output = 0;
prev_error = error;
```

7.4 PID vs Discrete Zone Override

The PID output is computed each iteration but is subsequently overridden by a discrete zone-based PWM assignment (Section 8). This design choice was intentional for the initial prototype: the zone-based approach provides deterministic, predictable behavior for demonstration purposes while the PID computation remains in the code as the basis for a future closed-loop implementation without zone overrides.

The PID output is still clamped and stored, meaning its integral state evolves throughout operation. In a future revision, the PID output would be used directly (without zone override) to achieve truly continuous proportional fan speed control.

8. Temperature Zone Logic & LED Control

8.1 Zone Definition

Three discrete temperature zones are defined, each mapped to a fixed PWM duty cycle and LED color. The thresholds were selected to provide a visible step response across the normal indoor temperature range:

Zone	Condition	PWM Value	Duty Cycle	LED	GPIO Pin
Low	$T < 26\text{ }^{\circ}\text{C}$	300	30%	Green	PD12
Medium	$27\text{ }^{\circ}\text{C} \leq T \leq 28\text{ }^{\circ}\text{C}$	600	60%	Yellow	PD13
High	$T > 28\text{ }^{\circ}\text{C}$	900	90%	Red	PD14

Note: There is an intentional hysteresis gap between the Low zone ($T < 26$) and Medium zone ($T \geq 27$). When temperature is in the 26–27 °C range, the code falls through to the High condition due to the else-if structure. This is a known behavior in the current implementation and would be corrected in a production system using explicit hysteresis bands.

8.2 LED Toggle Behavior

Each LED is toggled (not simply set high) on each loop iteration. Since the main loop executes every 1 second (HAL_Delay(1000) at end of loop), the active LED blinks at 0.5 Hz (1 second on, 1 second off). This blinking behavior provides a clear active indication and distinguishes the system from a frozen state. The two inactive LEDs are explicitly written low before the toggle to ensure only one LED blinks at any time.

8.3 Code Structure

```
HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12|GPIO_PIN_13|GPIO_PIN_14, GPIO_PIN_RESET);
if (temperature < 26) {
    pwm_output = pwm_low; // 300
    HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_12); // Green
} else if (temperature >= 27 && temperature <= 28) {
    pwm_output = pwm_mid; // 600
    HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_13); // Yellow
} else {
    pwm_output = pwm_high; // 900
    HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_14); // Red
}
```

9. Debugging Methodology & Verification

Beyond the SWV ITM telemetry described in Section 10, development relied on STM32CubeIDE's integrated debugger to verify peripheral behaviour at the register level and to catch logic errors before they surfaced as incorrect hardware behaviour. This section documents the breakpoint strategy, the register and variable inspection workflow, the ITM console verification process, and the cross-check performed against the oscilloscope.

9.1 Debug Configuration

Debugging was performed over the on-board ST-LINK/V2 SWD interface using the GDB-based debugger integrated into STM32CubeIDE. The debug session was launched via Run > Debug As > STM32 C/C++ Application, with the following debugger settings:

Setting	Value
Interface	SWD (Serial Wire Debug)
Probe	ST-LINK/V2 (on-board)
Reset Mode	Software system reset
Core Clock (for SWV)	84 MHz
SWV Enabled	Yes — ITM stimulus port 0

9.2 Breakpoint Strategy

Breakpoints were placed at four key points in main.c to isolate each stage of the control loop and confirm that data was correctly handed off between stages:

Location	Purpose
After HAL_ADC_GetValue()	Verify raw adc_val before voltage/temperature conversion
After temperature = voltage * 100.0f;	Confirm the LM35 scaling produced a sane temperature value
Inside the PID block, after pwm_output = ...	Inspect Kp/Ki/Kd contributions and the clamped output
Inside each zone branch (Low/Medium/High)	Confirm the correct branch was taken and the matching LED toggled

Because the main loop executes only once per second (HAL_Delay(1000)), single-stepping through one full iteration was practical without the target timing out or the watchdog (not used in this project) intervening. A conditional breakpoint was also used at the PID block, triggered only when temperature > 28.0f, to catch the High-zone transition without manually heating the sensor on every debug run.

9.3 Live Variable & Expression Inspection

With execution halted at a breakpoint, the STM32CubeIDE Variables view and a custom Live Expressions group were used to track the control loop's key state across iterations:

Variable	Typical Observed Value	What It Confirmed
adc_val	2980 – 3010	ADC conversion completing and returning a stable 12-bit code at room temperature
voltage	2.40 – 2.42	Voltage scaling formula ($\text{ADC_val} \times 3.3 / 4095$) matched expected LM35 output
temperature	24.0 – 24.2	LM35 $\times 100$ scaling correct; agreed with the reference thermometer
error / integral / derivative	varies	PID terms updating each iteration; integral accumulating without runaway growth
pwm_output	300 / 600 / 900	Zone override correctly replacing the raw PID output before the compare register update

Live Expressions were particularly useful for the integral term: by watching it accumulate in real time while the sensor was heated with the heat gun, it was possible to confirm the anti-windup clamp (Section 7.3) was capping pwm_output at pwm_max rather than allowing the integral term to grow unbounded, which directly informed the fix described in Section 12.

9.4 Peripheral Register Inspection

STM32CubeIDE's Peripheral Registers view (Window > Show View > SFRs, populated from the SVD file for STM32F407VGT6) was used to inspect peripheral state directly, bypassing the HAL abstraction to confirm the underlying hardware configuration was correct:

Peripheral	Register	Field(s) Checked	Expected / Observed
ADC1	SR	EOC (End Of Conversion)	Set to 1 immediately after HAL_ADC_PollForConversion() returned
ADC1	DR	DATA[11:0]	Matched the value returned by HAL_ADC_GetValue() (e.g. 0xBB4 $\approx$ 2996)
ADC1	CR2	ADON, CONT	ADON = 1 (ADC enabled); CONT = 0 (single-conversion mode, as configured)
TIM2	CCR1	Capture/Compare 1 value	Matched the last value written via __HAL_TIM_SET_COMPARE() (300 / 600 / 900)
TIM2	ARR	Auto-reload value	999, confirming the 1 kHz PWM period (1 MHz / 1000)
TIM2	CR1	CEN (Counter Enable)	CEN = 1, confirming the timer was running after HAL_TIM_PWM_Start()
GPIOD	ODR	ODR12, ODR13, ODR14	Only one bit high at a time, toggling at ~ 1 Hz, matching the active zone LED

Peripheral	Register	Field(s) Checked	Expected / Observed
GPIOD	MODER	MODE12, MODE13, MODE14	01 (General purpose output), confirming push-pull output mode

Reading TIM2_CCR1 directly in the register view was the most reliable way to confirm a PWM duty-cycle change had actually taken effect before reaching the oscilloscope, since it removed any uncertainty about whether the HAL call had written to the correct channel.

9.5 SWV ITM Console Verification

The SWV ITM Data Console (Window > Show View > SWV > SWV ITM Data Console) was used to confirm that printf telemetry was both correctly formatted and correctly timed:

- ITM stimulus port 0 was enabled and the console's record button activated before resuming execution, as described in Section 10.3.
- Each printed line was checked against the values held in the Variables view at the same breakpoint, confirming the printf format string (Section 10.1) reported the same temperature, ADC value, voltage, error, and PWM figures the debugger saw internally.
- The 0.1 °C change-detection threshold was verified by watching the console during a slow, stable-temperature period: no new line appeared until prev_temperature and temperature genuinely diverged by more than 0.1 °C, confirming the suppression logic in Section 10.1 worked as intended rather than printing on every loop iteration.
- During the early stage of development, the console initially produced no output at all; this was traced using the debugger to a missing _write() retarget and is documented as the first entry in the Challenges & Solutions table (Section 13).

9.6 Oscilloscope Cross-Check

To close the loop between the debugger's view of the register state and the actual electrical signal, the TIM2_CH1 PWM output on PA5 was probed with an oscilloscope while simultaneously halting execution on a breakpoint immediately after each zone's __HAL_TIM_SET_COMPARE() call:

Zone	CCR1 (Register View)	Oscilloscope High Time	Period	Result
Low	300	~300 μ s	1.00 ms (1 kHz)	Match
Medium	600	~600 μ s	1.00 ms (1 kHz)	Match
High	900	~900 μ s	1.00 ms (1 kHz)	Match

Because the high time observed on the oscilloscope matched the CCR1 value read from the register view in every zone, this confirmed both that the firmware was writing the intended compare value and that the TIM2 prescaler/ARR configuration from Section 5.3 was producing the correct 1 kHz carrier frequency on real hardware, not just in the debugger's model of the peripheral.

9.7 Summary of Debugging Workflow

In practice, each suspected issue was triaged using the same four-step sequence:

- Set a breakpoint at the relevant stage of the control loop (ADC read, temperature conversion, PID computation, or zone branch).
- Inspect the relevant variables and live expressions to localize whether the issue was in computation or in data hand-off between stages.
- Cross-check the corresponding peripheral register (ADC1, TIM2, or GPIOD) to rule out a HAL configuration mismatch versus an application-logic bug.
- Where the issue involved a physical signal (PWM duty cycle, LED timing), confirm the fix on the oscilloscope or visually before closing it out.

This combination of breakpoints, register inspection, and ITM-based telemetry verification, cross-checked against the oscilloscope, was directly responsible for catching three of the six issues listed in Section 13 (the missing `_write()` retarget, the fast LED blinking, and the integral windup) before they were misattributed to a hardware fault.

10. UART Serial Monitoring

10.1 Telemetry Format

A printf statement transmits a telemetry line over SWV ITM whenever the temperature changes by more than 0.1 °C since the last reading. This threshold-based reporting prevents flooding the console with identical readings during stable temperature periods.

```
if ((temperature - prev_temperature > 0.1f) ||
    (prev_temperature - temperature > 0.1f)) {
    printf("Temp: %.2f | ADC: %lu | Volt: %.2f | Err: %.2f | PWM: %.0f\r\n",
          temperature, adc_val, voltage, setpoint - temperature, pwm_output);
}
```

10.2 Sample Output

Field	Example Value	Description
Temp	24.75	Measured temperature in degrees Celsius
ADC	3068	Raw 12-bit ADC value (0–4095)
Volt	2.47	Computed analog voltage in Volts
Err	0.25	PID error = setpoint – temperature
PWM	300	Current PWM compare value applied to TIM2

10.3 SWV ITM Setup

To view SWV output in STM32CubeIDE: open Run > Debug Configurations > Debugger tab > enable Serial Wire Viewer, set Core Clock to 84 MHz. Then in the IDE, open Window > Show View > SWV > SWV ITM Data Console. Enable ITM stimulus port 0 and click the red record button before resuming execution.

11. Software Architecture & Code Walkthrough

11.1 File Structure

File	Role
Core/Src/main.c	Application entry point; all user logic, peripheral initialization, control loop
Core/Inc/main.h	Pin definitions, peripheral handle declarations
Core/Src/stm32f4xx_hal_msp.c	HAL MSP initialization hooks (GPIO alternate function, clock enables)
Core/Src/stm32f4xx_it.c	Interrupt service routines (SysTick, USART, etc.)
Drivers/STM32F4xx_HAL_Driver/	STMicroelectronics HAL library source

11.2 Main Loop Flowchart (Described)

The main() function performs the following steps in order after peripheral initialization:

- Start TIM2 PWM on Channel 1.
- Enter infinite while(1) loop.
- ADC: Start conversion, delay 5 ms, poll for result, compute voltage and temperature.
- UART: If temperature changed by >0.1 °C, print telemetry via SWV ITM.
- PID: Compute error, update integral, compute derivative, calculate raw PID output, clamp.
- GPIO: Reset all LEDs.
- Zone Logic: Assign discrete PWM and toggle the appropriate LED.
- Update TIM2 compare register with the new PWM value.
- Save prev_error for next iteration.
- Delay 1000 ms.

11.3 Variable Summary

Variable	Type	Description
adc_val	uint32_t	Raw 12-bit ADC conversion result
voltage	float	Computed analog voltage from ADC (V)
temperature	float	Computed temperature from LM35 (°C)
setpoint	float	Desired temperature target (25.0 °C)
Kp, Ki, Kd	float	PID gains (5.0, 0.1, 1.0)
integral	float	Accumulated error for integral term
prev_error	float	Error value from previous iteration (for derivative)
pwm_output	float	Computed (and clamped) PWM duty cycle value

Variable	Type	Description
pwm_max	uint32_t	Maximum PWM value = TIM2 Period = 1000
prev_temperature	float	Temperature at last UART print (for change detection)

12. Testing & Results

12.1 Test Methodology

The system was tested by heating the LM35 sensor using a heat gun at a controlled distance and observing system response via LED indicators and SWV ITM console output. Three test scenarios were evaluated: stable low temperature, step increase to medium zone, and sustained high temperature.

12.2 Test Results Summary

Test Case	Input Temperature	Expected LED	Observed LED	Expected PWM	Observed PWM	Pass/Fail
Low Zone	< 26 °C	Green (blink)	Green (blink)	300	300	PASS
Medium Zone	27–28 °C	Yellow (blink)	Yellow (blink)	600	600	PASS
High Zone	> 28 °C	Red (blink)	Red (blink)	900	900	PASS
UART Log	Any (>0.1 change)	N/A	N/A	N/A	Logged correctly	PASS
Fan Speed	Proportional	N/A	N/A	Varies	Audibly varies	PASS

12.3 ADC Accuracy

With the LM35 at room temperature (~24 °C), the measured ADC value was approximately 2980–3000 corresponding to a voltage of ~2.40V. The computed temperature from the firmware was 24.0–24.1 °C, which agrees with a reference thermometer reading of 24.0 °C to within ± 0.2 °C. This is consistent with the LM35 specified accuracy of ± 0.5 °C.

12.4 PWM Signal Verification

The PWM output on PA5 was observed on an oscilloscope. At 30% duty cycle (pwm_output=300), the signal showed a high period of approximately 300 μ s and a total period of 1 ms (1 kHz), confirming the TIM2 configuration is correct. Fan speed audibly changed between three zones confirming MOSFET switching functionality.

13. Challenges & Solutions

Challenge	Root Cause	Solution Applied
printf not visible in console	_write not retargeted to ITM initially	Implemented _write() using ITM_SendChar(); enabled SWV in debug config
LED blinking too fast	Toggle called without delay in early version	Moved toggle inside 1-second loop; now blinks at 0.5 Hz visibly
Temperature reading drifting	No decoupling on LM35 Vs pin	Added 100 nF ceramic capacitor between Vs and GND near sensor
Fan not spinning at low PWM	MOSFET not fully enhancing at low gate voltage	Confirmed MOSFET is logic-level compatible with 3.3V gate drive
Zone gap at 26–27 °C	Else-if boundary missed 26–27 range	Documented as known issue; plan to refactor with explicit hysteresis bands
Integral windup during testing	Integral accumulated while sensor was heated	Anti-windup clamp added (pwm_output capped at 0 and pwm_max)

14. Conclusion & Future Work

14.1 Conclusion

This project successfully implements a complete embedded temperature-controlled fan regulation system on the STM32F407VGT6 platform. The system integrates ADC sensing, PID control logic, PWM-based actuation, GPIO LED indication, and UART/SWV telemetry into a cohesive embedded application developed in STM32CubeIDE using the HAL driver layer.

The prototype demonstrates correct operation across all three thermal zones, with reliable LED visual feedback and consistent fan speed variation. The ADC temperature measurement shows accuracy within ± 0.2 °C of reference, and the PWM output was verified on an oscilloscope. The project reinforces core embedded systems skills including peripheral configuration, interrupt-free polling-based control loop design, breakpoint- and register-level debugging, and hardware-software co-design on an evaluation board.

14.2 Future Work

- Replace discrete zone control with pure PID closed-loop for smooth, continuous fan speed regulation.
- Add interrupt-driven ADC using DMA for non-blocking sampling and higher sampling rates.
- Implement UART-based command interface to allow runtime adjustment of setpoint and PID gains.
- Add an OLED display over I2C (e.g., SSD1306) to show temperature and fan speed numerically.
- Integrate tachometer feedback from fan RPM signal for true closed-loop speed control.
- Explore CAN bus integration for multi-node thermal monitoring in a distributed system.
- Port the PID algorithm to use ARM CMSIS DSP library for optimized fixed-point computation.
- Add anti-windup integrator clamping tied to actuator saturation for production-grade PID implementation.

References

- [1] STMicroelectronics, “STM32F407xx Datasheet”, DS8626, Rev 9, 2023. Available: <https://www.st.com>
- [2] STMicroelectronics, “STM32F4xx HAL and Low-layer drivers — User Manual”, UM1725, Rev 9, 2023.
- [3] STMicroelectronics, “STM32F407G-DISC1 Discovery Kit User Manual”, UM1472, Rev 4, 2021.
- [4] Texas Instruments, “LM35 Precision Centigrade Temperature Sensor Datasheet”, SNIS159H, 2017.
- [5] K.J. Åström and T. Hägglund, “PID Controllers: Theory, Design, and Tuning”, 2nd ed., ISA, 1995.
- [6] J. Yiu, “The Definitive Guide to the ARM Cortex-M4”, 1st ed., Elsevier/Newnes, 2013.
- [7] STMicroelectronics, “STM32CubeIDE User Guide”, UM2609, 2023.
- [8] ARM Limited, “Cortex-M4 Technical Reference Manual”, ARM DDI 0439C, 2010.